Aveti Learning — Codex Build Runbook

Idea → working app, in the order you actually do it

This tells you exactly what to paste into Codex, what to run, and the few steps only you can do (creating accounts and handling keys — Codex must never do those). Build in phases; test after each.

Before you start (one-time, you do these)

- Install Node.js (LTS) and have Codex available in your terminal / IDE.
- Create three free accounts yourself: **GitHub**, **Supabase**, **Vercel**. (Codex must not create accounts or log into them on your behalf.)
- Put your three reference files in the project so Codex can read them: `aveti-learning-pm-case-study.md`, `aveti-report-app-build-spec.md`, `aveti-storage-addendum.md` → keep them in a `/docs` folder. Have your screenshots ready to attach.

> Rule for every Codex prompt: *"The build spec is the source of truth for behaviour and data; the screenshots are the visual reference. If they conflict, the spec wins."*

Phase 0 — Scaffold the project (Codex)

Open an empty folder `aveti-learning/` in your editor, start Codex in it, and paste:

> Scaffold a single Vite + React app in this folder. Add `@supabase/supabase-js` and `react-router-dom`. Create `src/lib/supabase.js` that initialises the Supabase client from `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` environment variables. Add a `.gitignore` that excludes `.env`, `.env.local` and `node_modules`. Add a `.env.example` with placeholder keys. Set up routes for Home, EnterMarks, TeacherReport, ParentShare, Growth, and a StudentRoster page, with empty placeholder components for now. Do NOT build the screens yet — just a skeleton with a working dev server. Then run the dev server so I can confirm it loads.

Confirm `http://localhost:5173` opens a blank skeleton before moving on.

Phase 1 — Set up Supabase (you + Codex)

You (in the Supabase dashboard):

1. Create a new project.
2. Open the SQL editor, paste the SQL from `docs/aveti-storage-addendum.md`, run it (creates the 4 tables + RLS).

3. Go to Project Settings → API. Copy the **Project URL** and the **anon public** key.
4. In your project folder, create a `.env` file and add:

```
VITE_SUPABASE_URL=...your project URL...
VITE_SUPABASE_ANON_KEY=...your anon key...
```

Never paste the `service_role` key anywhere in the app. Never commit `.env`.

Then Codex:

> Using the schema in `docs/aveti-storage-addendum.md`, create `src/lib/db.js` with data-access functions for centres, students, tests and results (create / read / update). Add a Supabase email-and-password auth gate: the app requires login, creates one `centres` row on first login, and all data is scoped to the logged-in centre. Add a simple login screen.

Test: sign up, confirm a `centres` row appears in Supabase.

Phase 2 — Build the MVP screens (Codex, in this order)

Attach the screenshots and reference the spec each time. Build and test one screen group at a time.

2a — Enter marks + Save (spec §4B, §4C):

> Build the StudentRoster page (set `parent_phone` per student, with country code) and the EnterMarks screen per spec §4B, plus the Save confirmation §4C. Use `src/lib/db.js`. Follow the attached screenshots for layout. Enforce spec §6: percentage is computed not stored; absent students are excluded from averages; block save if a student is blank and not absent; full-marks toggle 20/25 re-validates. Mobile-first.

2b — Teacher report (spec §4D):

> Build the Teacher report (§4D): class average, highest/lowest, A/B/C/D bands, ranked list with
> trend arrow and a "needs support" flag. Add a print stylesheet so it prints clean A4 with the centre letterhead. Match the screenshot.

2c — Parent share (spec §4E, manual WhatsApp):

> Build the Parent share screen (§4E). For each student show name, score and parent phone with an include checkbox and a privacy banner. Each Send button opens `https://wa.me/<phone>?text=<message>` with a pre-filled message that links to that student's hosted report-card page. No image auto-attach (that's a later Business-API phase). Flag students with no phone number and exclude them.

Stop here — this is your shippable MVP. (Growth tracker = next phase, once you have ≥2 tests of data.)

Phase 3 — Run & test locally

- `npm run dev`, then walk the acceptance checks in spec §8: enter 4 students incl. 1 absent → average excludes the absentee; switch 25→20 → values re-clamp; parent with no number is flagged and skipped; each report prints on one A4 page.

Phase 4 — Push to GitHub

You: create an empty **private** repo `aveti-learning` on github.com (don't add a README).
Then Codex:

> Initialise git, make a first commit, add the remote `https://github.com/<me>/aveti-learning.git` and push to `main`. Confirm `.env` is NOT in the commit.

(If you've set up the GitHub CLI and authenticated it yourself, you can instead tell Codex to use `gh repo create aveti-learning --private --source=. --push`.)

Phase 5 — Deploy to Vercel (you)

Codex can't log into your Vercel — do this yourself:

1. vercel.com → Add New → Project → import the `aveti-learning` repo.
2. Framework auto-detects as Vite (build `npm run build`, output `dist`).
3. Settings → Environment Variables: add `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` (same values as your `.env`).
4. Deploy → you get a live URL.
5. In Supabase → Authentication → URL Configuration, add your Vercel URL to allowed/redirect URLs (or login breaks).

The iteration loop (forever after)

Edit with Codex in VS Code → `git push` → Vercel auto-redeploys. That's your whole release process.

How to get good output from Codex

- Build in small phases and **test after each** — don't ask for the whole app in one prompt.
- Always point it at the spec as source of truth and attach the relevant screenshot.

- When something's off, paste the exact behaviour you saw, not "it's broken."
- Let Codex run the dev server and check its own work, then you verify against §8.
- Keep keys in `.env` only; if Codex ever suggests hardcoding a key, say no.

Phases after the MVP (don't build yet)

Growth tracker → Odia/Hindi cards → bulk WhatsApp Business API → teacher analytics for the owner.
Each is a fresh, small Codex task on top of the same repo.